Worksheet 4: Pointers, functions and structures

ECM2433: The C Family

Spring 2026

1 Git

You should be able to get this worksheet and the necessary files by `git pull` from your clone of the module repository. This will also put possible answers for last week's workshop in `.../week03/possible_answers/`.

2 Address sanitisation

One of the hazards of C and especially using pointers in C is that we may try to access illegal memory addresses or memory that we haven't allocated or freed. `valgrind` is a very useful tool for detecting these mistakes. An alternative is *address sanitisation*. This means that the compiler (`gcc` for us) puts "guards" around arrays and `malloc`-ed memory to perform checks at runtime. It comes with a performance cost and you program will run a bit slower, but for the programs we are writing, the difference will be unnoticable. For larger programs, you can get the bugs out using address sanitisation and then compile the production version without the sansitisation flags.

Here's an example, `corrupt.c`, in which we set `str2[-16] = 'x'` which is outside the range of legal addresses

```
#include <stdio.h>

int main()
{
    char str1[20] = "Goodbye";
    char str2[6]  = "Hello";
    int j = -16;
    str2[j] = 'x';
    printf("str1: %s\n", str1);
```

```
    printf("str2: %s\n", str2);

    return 0;
}
```

This compiles and runs without a problem in this instance. We can compile it for address sanitisation as follows:

```
$ gcc -o corrupt -fsanitize=address -O1  -g  corrupt.c
```

The `-O1` means use the lowest level of optimisation and the `-g` keeps information about the source which allows better error messages to be given. The result is

```
[reverson@mcrugcomp01 ~]$ gcc -o corrupt -fsanitize=address -O1  -g  corrupt.c
[reverson@mcrugcomp01 ~]$ ./corrupt
=================================================================
==2540561==ERROR: AddressSanitizer: stack-buffer-underflow on address 0x7ffdf3902e20 at pc 0x000000401372 bp 0x7f
fdf3902e00 sp 0x7ffdf3902df0
WRITE of size 1 at 0x7ffdf3902e20 thread T0
    #0 0x401371 in main /home/links/reverson/corrupt.c:8
    #1 0x7f770f0295cf in __libc_start_call_main (/lib64/libc.so.6+0x295cf)
    #2 0x7f770f02967f in __libc_start_main@GLIBC_2.2.5 (/lib64/libc.so.6+0x2967f)
    #3 0x4010d4 in _start (/home/links/reverson/corrupt+0x4010d4)

Address 0x7ffdf3902e20 is located in stack of thread T0 at offset 16 in frame
    #0 0x4011a5 in main /home/links/reverson/corrupt.c:4

  This frame has 2 object(s):
    [32, 38) 'str2' (line 6) <== Memory access at offset 16 underflows this variable
    [64, 84) 'str1' (line 5)
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext or vfork
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-underflow /home/links/reverson/corrupt.c:8 in main
Shadow bytes around the buggy address:
  0x10003e718570: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x10003e718580: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x10003e718590: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x10003e7185a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x10003e7185b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x10003e7185c0: 00 00 f1 f1[f1]f1 06 f2 f2 f2 00 00 04 f3 f3 f3
  0x10003e7185d0: f3 f3 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x10003e7185e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x10003e7185f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x10003e718600: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0x10003e718610: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Shadow byte legend (one shadow byte represents 8 application bytes):
  Addressable:           00
  Partially addressable: 01 02 03 04 05 06 07
  Heap left redzone:       fa
  Freed heap region:       fd
  Stack left redzone:      f1
  Stack mid redzone:       f2
  Stack right redzone:     f3
  Stack after return:      f5
  Stack use after scope:   f8
  Global redzone:          f9
  Global init order:       f6
  Poisoned by user:        f7
  Container overflow:      fc
  Array cookie:            ac
  Intra object redzone:    bb
  ASan internal:           fe
  Left alloca redzone:     ca
  Right alloca redzone:    cb
  Shadow gap:              cc
==2540561==ABORTING
[reverson@mcrugcomp01 ~]$
```

There are extensive messages and analysis, but in particular the problem (with `str2`) and the line (4) at which the problem occurred is identified.

In the last workshop you write an `intArray` program and then modified it to use dangling and wild pointers, and free-ing `malloc`-ed memory more than one and freeing an array that was declared rather than `malloc`-ed. Repeat these but compiling with address sanitisation to see the result.

In general, **there's no reason not to compile using address sanitisation while you're developing a program;** turn it off when everything is working.

3 Pass by value and reference

By value

Write a function called `printInteger` that takes a single integer as a parameter, prints out its value and adds 3 to it. The return value of the function should be the updated value of the integer. Call your function like this:

```
int i = 42;
int j = 0;

printf("in main function i = %d, j = %d\n", i, j);
j = printInteger(i);
printf("in main function i = %d, j = %d\n", i, j);
```

The value of `y` should be 0 from the first `printf` and 45 from the second. This is done for you in `test-printInterger.c` and `test-printInteger.toml`: you will have to make sure that your function prints with the format string:

```
"in printInteger i = %d\n"
```

for the test to pass.

By reference

Write another function called `updateInteger` that updates the value of the integer whose address is passed to it by adding 3 to it. It should also print the value of `*i` before altering its value, using the format string `"in updateInteger deferenced pointer = %d\n"`. Note that parameter will have to be a pointer to the integer in order for the value to be able to be updated. Call your function like this:

```
    int i = 42;

    printf("in main function i = %d\n", i);
    updateInteger(&i);
    printf("in main function i = %d\n", i);
```

You can test it with `test-updateInteger.toml`.

4 Swapping

Write and test a function `iswap(int *p, int *q)` that swaps the values of `*p` and `*q`. Thus

```
int i = 5;
int j = 2;

swap(&i, &j);
printf("i = %d j= %d\n", i, j);
```

should result in

```
i = 2 j = 5
```

being printed.

Extend your function to swap anything instead of just integers. The function should have prototype:

```
void swap_anything(void *p, void *q, size_t size);
```

where `size` is the size (in bytes) of the variables that `p` and `q` point to. *Hint:* You may need to `malloc` (and `free`) temporary storage in the function.

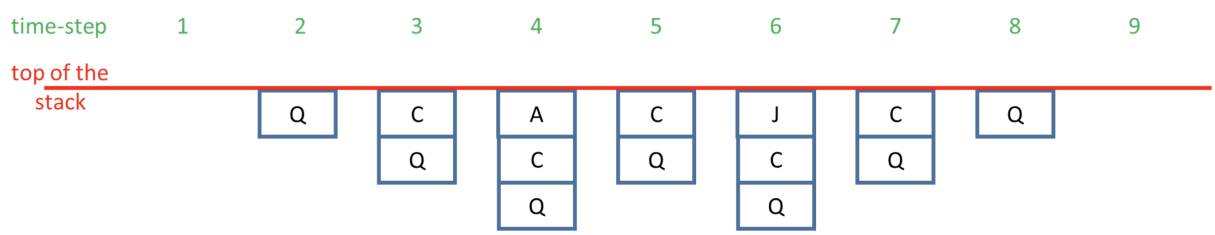
If your code is in a file `swap.c`, the program `test-swap.c` and `test-swap.toml` can be used to test your function on a variety of types.

5 Structures and stacks

In this exercise you are going to write a program to implement a stack using C structures and pointers. A stack is a Last In First Out (LIFO) queue, with two key operations that manipulate its contents:

push add an item to the top of the stack
pop remove the top-most item from the stack (the one most recently "push"ed)

Here is an example of how a stack works. The stack is in blue, and at the start (step 1, labelled in green) it is empty. If we `push` the value 'Q' onto the list, the resulting stack is shown as step 2. Pushing 'C' and then 'A' results in the stack labelled 3 and 4 respectively. Now performing a `pop` removes the top element of the stack, i.e. the value 'A' (step 5). The remaining operations are push 'J' (step 6), pop (step 7), pop (step 8), and, finally, pop (step 9), resulting in an empty stack.



As well as `push` and `pop` functions, you should implement a function called `print_stack` for printing the current state of the stack. If, for example, you call this function when the stack is as shown in step 4, then the result should be `ACQ`. Note that you can implement this using a while loop; a more elegant, but less efficient, solution is a recursive function.

If you do not know how to start to tackle this problem, then I suggest you approach it in the following order:

5.1 Key code

Get the key code working:

1. Define a structure (for example, called `stackStruct`) to represent a single node in the stack. Use `typedef` to create a synonym for `struct stackStruct` so that you can refer to the datatype `STACK`.
2. Initialise an empty stack.
3. Create a function that creates a new node. The function prototype should be defined as

```
STACK *newNode(char data);
```

You can call it, for example, to initialise a variable, like this: `STACK *s = new_node('Q');`

4. In the main function, create a new node (with value 'Q') and push it onto the stack.
5. In the main function, create a new node (with value 'C') and push it onto the stack.
6. Create the `print_stack` function and use it print out the current state of the stack.
7. Print out the current state of the stack.
8. In the main function, pop the top node off the stack and print out its value (i.e. 'A').

You could do all these in a single file.

5.2 Re-organise your code

Create functions called `push` and `pop` with prototypes

```
STACK *push(STACK *s, char data);
char pop(STACK **s);
```

Here `push` should return the updated stack. Pop should return the data and the modified stack via `**s`. Put these, along with your `new_node()` and `print_stack()` functions in a file `stack.c` and put the driver code in a separate file. Check that your functions work. Make sure that the stack is completely empty before the main program ends and use address sanitisation and/or valgrind to check.

Make sure that your code can cope with edge cases: for example, popping an empty stack.

5.3 Generalise

The code you have written so far stores a single character in each node of your stack. It would be more useful to be able to store arbitrary data in each node. One way to accomplish this is to use a pointer to a void. So, for example, the node could be declared as:

```
typedef struct stackStruct {
     void *data;
     struct stackStruct *nextNode;
} STACK;
```

The `*data` pointer can then be used to point to anything – a string or a pointer to an integer or float, or a pointer to another struct containing several items. Modify your code to incorporate this generality. Remember that your code that uses the data will have to cast the void pointer to the appropriate type.

5.4 Prime Ministers

Use your stack to write a program that builds a stack of the British prime ministers, their terms of office and political parties using the data in the file `BritishPM.txt`. You should create a new struct to hold this data about each PM.

The function `scanf()` is used for formatted reads from the terminal. However, you can get it to read from a file as follows:

```
$ myProgram < BritishPM.txt
```

In this case `scanf` will read from your file until it is exhausted instead of from the terminal.

Each line of `BritishPM.txt` is formatted similar to the following:

```
1868 - 1868 Benjamin Disraeli   Conservative
```

Here the dates are separated from the name by a tab (''$\widehat{}$) character and the name is separated from the party by a second tab. An attractive option would be to use `scanf` with a format string like this:

```
int start, end;
char who[31], party[41];

scanf("%4d - %4d\t%s\t%s\n", &start, &end, who, party);
```

which specifies that the strings are separated by tab characters. However, if the name were longer than 30 characters or the party longer than 40 characters, this could lead to the read string overflowing the space allocated for it. A better format string is:

```
scanf("%4d - %4d\t%30[^\t]\t%40[^\n]\n", &start, &end, who,
   ↪ party);
```

which specifies that the `who` string consists of up to 30 non-tab characters and the `party` string consists of 40 non-newline characters. An example program showing how to read the file is `readPM.c`; run it as

```
$ readPM < BritishPM.txt
```

Finally, pop the prime ministers off the stack and print them out to obtain a list of the prime ministers, their terms of office and parties in reverse chronological order. Use `valgrind` and/or address sanitisation to make sure you have no memory leaks!

6 Merge sort

Merge sort works by recursively dividing an array into two, sorting each half and merging the two sorted halves in order. See, for example, the Wikipedia page.

The file `mergesortInt.c` contains an implementation of merge sort for arrays of integers and `test-mergesortInt.c` is a little program to test it. You can use `../tester/tester.py test-mergesortint.toml` to test this, but look at what the program does too.

The goal of this exercise is to convert the integer array implementation into a merge sort that will sort arrays of any type. To do this we need to

1. Pass the arrays to be sorted as `void *`
2. Supply a comparison function that accepts two `void *` pointers to elements of the array and compares the elements of the array that they point to. The prototype is:

   ```
   int (*compare)(void *p, void *q)
   ```

   and the returned integer should be negative, zero or positive if the array element pointed to by `p` is less than, equal to or greater than the element pointed to by `q`. Recall the example with `qsort` in the slides.

3. Rather than accessing array elements directly (using `[]`), use pointers to the appropriate memory location and copy the appropriate number of bytes. For example, suppose that `void *v` is a pointer to an array that we know is an array of whose elements are `width` bytes, then we could copy the `i` th element to the `j` th element with:

   ```
   char *array = (char *)v;      // Pointer to char for
                                 //  pointer arithmetic
   char *ip = array + i*width;   // Points at the ith element
   char *jp = array + j*width;

   for (int n = 0; n < width; n++)
        *jp++ = *ip++;           // Copy width bytes
   ```

Note that the loop could be replaced with a call to `memcpy` that copies a given number of bytes; see `man memcpy`. Strictly, we have to cast `v` to `char *` before doing pointer arithmetic on it, but GCC allows pointer arithmetic on `void *` directly.

Write a merge sort function `merge_sort`[1] modelled on `mergesortInt` to accept any array. It should have prototype

```
void merge_sort(void *array, size_t width,
                int ileft, int iright,
                int (*compare)(const void *p, const void *q));
```

The file `test-mergesort.c` contains code to test your function on arrays of ints and strings and you can use `test-mergesort.toml` to run tests with `tester.py`.

Bonus: Use your `merge_sort` function in a program to print the British prime ministers in alphabetical order of their names together with their parties and terms of office. Count "Sir Robert Walpole" as a name beginning with 'S' and "Earl of Wilmington" as a name beginning with 'E'.

[1] It's not called `mergesort` to avoid clashing with one in the C library.